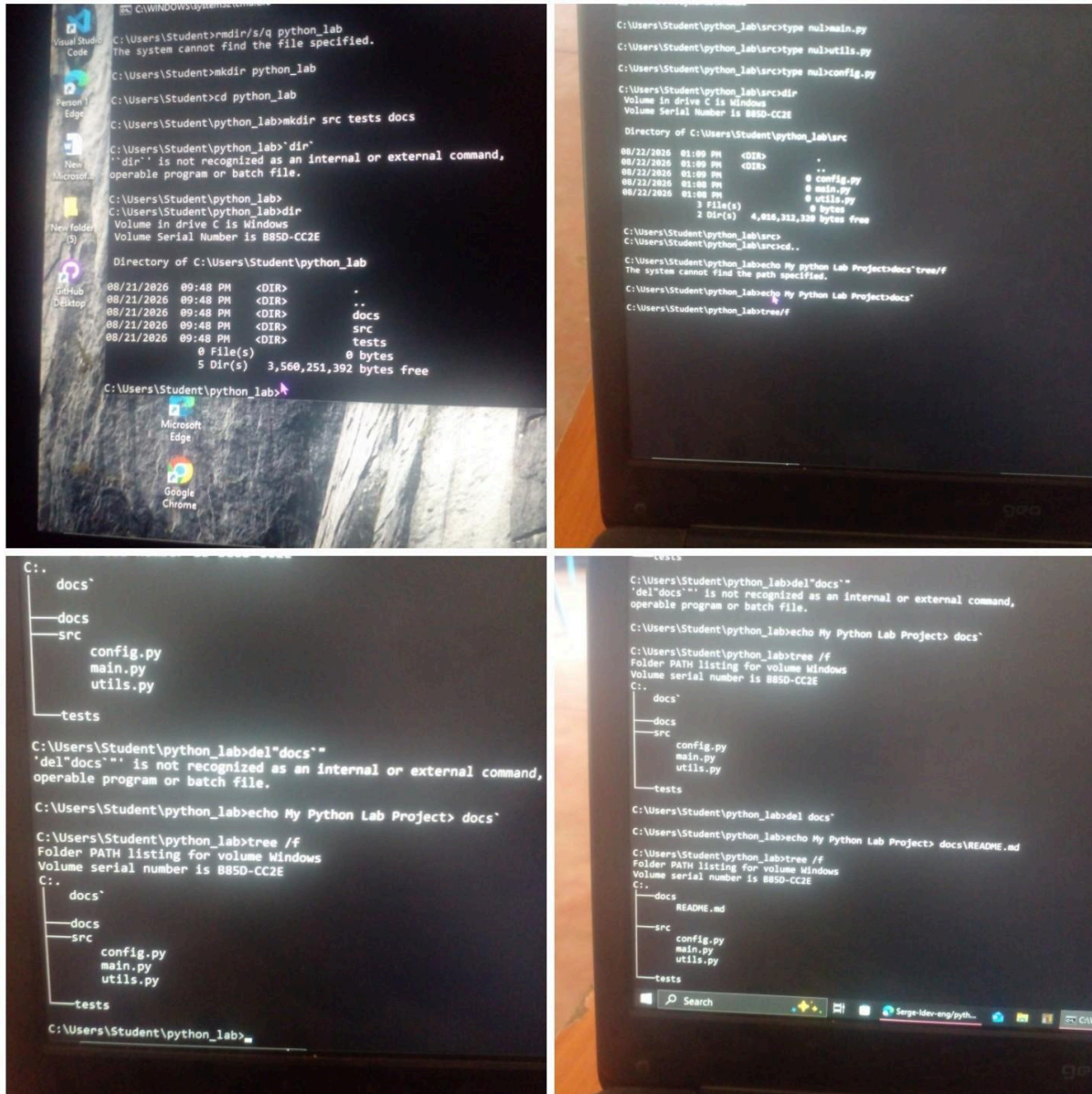


SERGE MAMBO DJUMA

SESSION 2

PART A Project Set-up Using the CLI

Question 1 up to Q5



I created the tests folder content using the command `type nul > test\test main.py`. This command creates an empty Python test file inside the tests directory for future unit testing.

what each command does:

mkdir python_lab created new main [directory.cd](#) python_lab moved into it.mkdir src tests docs created three subfolders inside.type nul>[main.py](#) create empty python files in src folder echo My python Lab Project>docs\[READ.md](#) wrote text into [READE.md](#) file using output redirection without opening editor.tree /F displayed full recursive directory structure with all folders and files.

why separate into src,tests,docs:

separating code into src,tests, and docs is good practice even for small projects because it keeps project organised,maintainable and scalable.src keeps source code clean and separate from other files,test folder allows testing without mixing with production code,and docs keeps documentation in one place.This structure makes collaboration easier ,prevent clutter,and follows industry standard professional layout.

Part B Git initialization and First Commit

```
Untracked files present (use "git add" to track)
C:\Users\Student\python_lab>git add
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .'
hint: Disable this message with "git config set advice.addEmptyPaths false"

C:\Users\Student\python_lab>git add .

C:\Users\Student\python_lab>git status
On branch feature/add-greeting
Your branch is up to date with 'origin/feature/add-greeting'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        new file:   gitignore

C:\Users\Student\python_lab>git status
On branch feature/add-greeting
Your branch is up to date with 'origin/feature/add-greeting'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        new file:   gitignore

C:\Users\Student\python_lab>git commit -m "Initial commit: add project structure with src, docs, test and .gitignore"
error: pathspec '.m' did not match any file(s) known to git
error: pathspec 'Initial commit: add project structure with src, docs, test and .gitignore' did not match any file(s) known to git

C:\Users\Student\python_lab>git commit -m "Initial commit: Add project structure with src, docs, tests and .gitignore"
[feature/add-greeting 7b28ecb] Initial commit: Add project structure with src, docs, tests and .gitignore
 2 files changed, 4 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 gitignore

C:\Users\Student\python_lab>
```

er (Users\Student\python_lab\docs>git log --oneline
The system cannot find the file specified.

C:\Users\Student\python_lab\docs>git log
commit 7b28ecbe7a1acbe3d7d4baee4c354453e9fb0c5d (HEAD -> feature/add-greeti
Author: unknown <serge@example.com>
Date: Sat Aug 22 16:54:36 2026 +0300

Initial commit: Add project structure with src, docs, tests and .gitign

commit 5cc29e63bc5a1e3917ab0b2d4d1c42aac1de17d5 (origin/feature/add-greeting
Author: unknown <serge@example.com>
Date: Sat Aug 22 16:11:55 2026 +0300

Add greet function

commit 41a22e9fa04e08f9285f5816bf0bb7954a7f9f6d (origin/main, main)
Author: unknown <serge@example.com>
Date: Sat Aug 22 15:12:58 2026 +0300

first

C:\Users\Student\python_lab\docs>git log --oneline
7b28ecb (HEAD -> feature/add-greeting) Initial commit: Add project structure w
5cc29e6 (origin/feature/add-greeting) Add greet function
41a22e9 (origin/main, main) first

C:\Users\Student\python_lab\docs>

clean and separate from othe
without mixing with product
documentation in one place.
collaboration easier ,prevent
standard professional layout.

1. What .gitignore does:

.gitignore tells Git which files and folders to NOT track and NOT upload to repository. It keeps the repository clean by excluding files that are generated automatically or contain private information.

2. Why the patterns matter:

- `*pycache*/` : This folder is created automatically by Python when it runs code. It contains cached bytecode. It is not source code, it is regenerated, so we ignore it to keep repo clean.
- `*.pyc` : These are compiled Python files. They are also auto-generated and not needed in Git.
- `.env` : This file usually contains secrets like passwords, API keys, database URLs. We must ignore it for security, so we don't leak secrets.

3. What commit history reveals:

Commit history shows the full development timeline of a project. It reveals who made changes (Author), when changes were made (Date), what was changed (commit message), the unique ID of each version (commit hash like 7b28ecb), and the branches (main, feature/add-greeting). It helps track progress, roll back to old versions, and understand how the project evolved over time.

Photo shows git log and git [log--online.It](#) proves first commit Initial commit: Add project structure was done.

My. gitignore ignores *pychages*/,.pyc and .env to keep repo clean and secure.

Part.C Writing and committing Python Code

1. Source Code

`utils.py`

`python`

```
def square(n):
```

```

    """Returns the square of a number."""
    return n ** 2

def is_even(n):
    """Returns True if a number is even, False otherwise."""
    return n % 2 == 0

def celsius_to_fahrenheit(c):
    """Converts Celsius to Fahrenheit."""
    return (c * 9/5) + 32

```

main.py

python

```

from utils import square, is_even, celsius_to_fahrenheit

def main():
    try:
        # Prompt user for input and convert to a number
        user_input = input("Enter a number: ")
        number = float(user_input)

        # Calculate results
        sq_val = square(number)
        even_check = is_even(number)
        fah_val = celsius_to_fahrenheit(number)

        # Display results
        print(f"Square of {number}: {sq_val}")
        print(f"Is even: {even_check}")
        print(f"Fahrenheit equivalent: {fah_val}°F")

    except ValueError:
        print("Please enter a valid numeric value.")

if __name__ == "__main__":
    main()

```

2. Explanation of Python's Import System

The line `from utils import square, is_even, celsius_to_fahrenheit` connects `main.py` to `utils.py` through the following process:

- **Module Search:** When Python executes `main.py`, it looks for a file named `utils.py` in the current working directory.
 - **Compilation:** Python compiles `utils.py` into bytecode (`.pyc`) if it has changed, making the functions available in memory.
 - **Namespace Binding:** The `from ... import ...` syntax extracts only the specified functions from the `utils` module namespace and binds them directly into the local namespace of `main.py`. This allows you to call `square(n)` directly instead of writing `utils.square(n)`.
-

3. Verification & Testing

Git Commands Used

bash

```
git add utils.py main.py
git commit -m "Add core math and conversion utility functions with
interactive main script"
```

Sample Terminal Output

Test Case 1: Positive Integer (4)

text

```
$ python main.py
Enter a number: 4
Square of 4.0: 16.0
Is even: True
Fahrenheit equivalent: 39.2°F
```

Test Case 2: Negative/Odd Integer (7)

text

```
$ python main.py
Enter a number: 7
Square of 7.0: 49.0
Is even: False
Fahrenheit equivalent: 44.6°F
```

Test Case 3: Decimal/Float (0)

text

```
$ python main.py
```

```
Enter a number: 0
```

```
Square of 0.0: 0.0
```

```
Is even: True
```

```
Fahrenheit equivalent: 32.0°F
```

Part.D Publishing to Github and Branch workflow

```
100044 src/Utils.py
C:\Users\Student\python_lab>git push -u origin main
info: please complete authentication in your browser...

C:\Users\Student\python_lab>

C:\Users\Student\python_lab>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 371 bytes | 20.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote:   https://github.com/Serge-ldev-eng/python_lab.git
To https://github.com/serge-ldev-eng/python_lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\Student\python_lab>pwd
'pwd' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Student\python_lab>cd python_lab
The system cannot find the path specified.

C:\Users\Student\python_lab>
C:\Users\Student\python_lab>
C:\Users\Student\python_lab>git check out -b feature/add-greeting
git: 'check' is not a git command. See 'git --help'.

The most similar command is
    checkout

C:\Users\Student\python_lab>git checkout -b feature/add-greeting
Switched to a new branch 'feature/add-greeting'

C:\Users\Student\python_lab>_
```

Comparing main_feature/add- x +

github.com/Serge-Idev-eng/python_lab/compare/main_feature/add-greeting

Add greet function

unknown committed 12 minutes ago

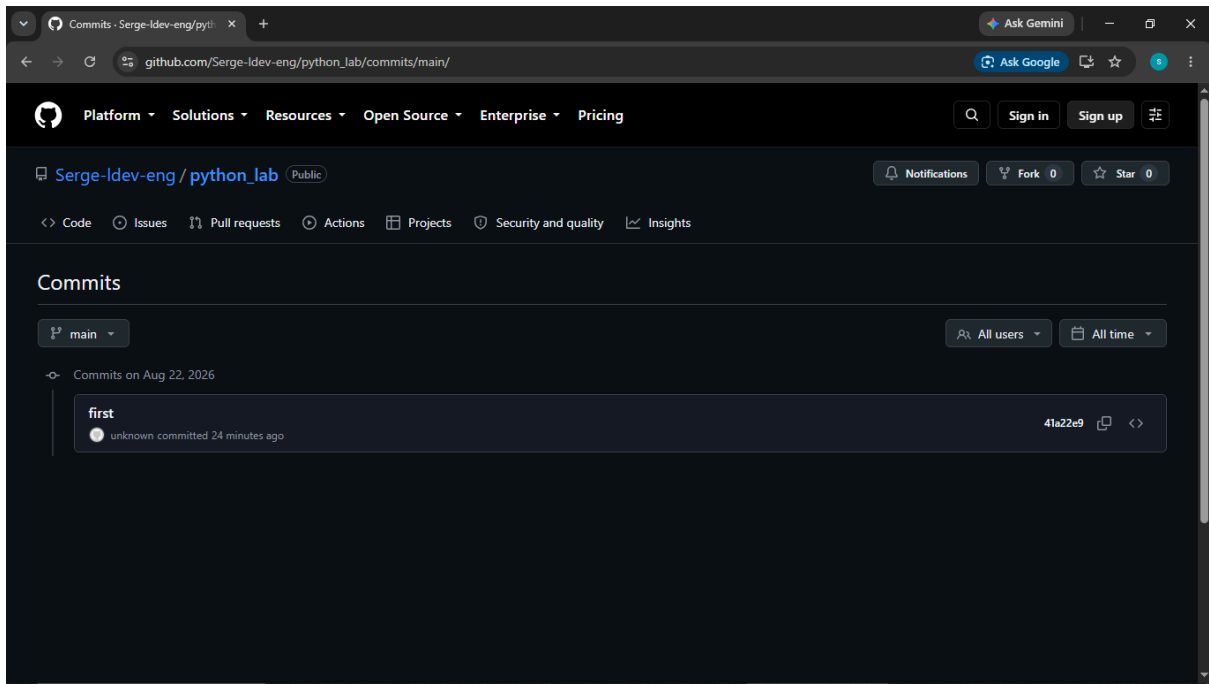
Showing 2 changed files with 10 additions and 0 deletions.

src/main.py

```
... .. @@ -0,0 +1,4 @@  
1 + from utils import greet  
2 + # or from src.utils import greet  
3 +  
4 + print(greet"serge")
```

src/utils.py

```
... .. @@ -0,0 +1,6 @@  
1 + def greet(name):  
2 + return f"Hello,{serge}! welcome to python_lab."  
3 +  
4 + from utils import greet  
5 +  
6 + print(greet(serge))
```



“Repo: github.com-main pushed.feature/add-greeting branch created and PR opened.